

wxsum: A command for processing temperature and precipitation data

Jeffrey D. Michler	Anna Josephson	Oscar Barriga Cabanillas
University of Arizona	University of Arizona	World Bank
Tucson, AZ	Tucson, AZ	Washington, D.C.
jdmichler@arizona.edu	jdmichler@arizona.edu	obarriga@worldbank.org
Aleksandr Michuda	Jeffrey C. Oliver	
Swarthmore College	University of Arizona	
Swarthmore, PA	Tucson, AZ	
amichud1@swarthmore.edu	jdmichler@arizona.edu	

Abstract. In this article, we introduce the community-contributed command `wxsum` which processes daily temperature and precipitation data and outputs useful summary statistics as well as measures of weather “shocks.” The command calculates 18 different measures of precipitation and 13 different measures of temperature. These include simple summary statistics, like the mean and standard deviation, as well as measures of shocks, such a z-score, dry spells, and killing degree days (KDD). The goal of the command is to reduce the cost of using weather data in socioeconomic analysis by allowing for quick and accurate calculations of the most common weather metrics used in the literature.

Keywords: st0001, `wxsum`, Earth observation, weather data, summary statistics

1 Introduction

The expanding availability and the decreasing costs of use have made weather data, particularly remotely sensed data, a common input into socioeconomic analysis (Dell et al. 2014; Donaldson and Storeygard 2016). Economists use these ready-made Earth observation products to predict a variety of economics outcomes, such as agricultural output, industrial output, labor productivity, energy demand, health, conflict, and economic growth. But, there are many ways to quantify weather and very little theoretical or systematic guidance on which weather metric is best suited to a given research question and geographic context. The results is that seemingly similar measures of “rainfall,” “temperature,” or “shock” can be based on very different summaries of the underlying weather data (Michler et al. 2021). The variety of metrics used and the size of weather data frames can create data handling and cleaning problem, meaning that the probability of making a coding error in calculating any given metric is non-trivial.

In this article, we introduce the community-contributed command `wxsum` which processes daily temperature and precipitation data and outputs useful summary statistics as well as measures of weather “shocks.” The goal of the command is to allow for easy and error free calculation of the most common weather metrics used in the socioeconomic literature. The `wxsum` command takes in daily temperature or precipitation data,

typically in the form of gridded Earth observation data, and calculates 18 different measures of precipitation and 13 different measures of temperature. These include a number of summary statistics for both temperature and precipitation, like the mean, median, standard deviation, and skew. It also calculates phenomenon-specific statistics, such as max seasonal temperature, growing degree days (GDD), and total seasonal rainfall. The command also calculates a number of measures of shocks. These include deviations from long-run averages, z-scores, dry spells, and killing degree days (KDD).

The **wxsum** command allows the user to set a variety of options so as to customize the weather metrics to their specific geography and research context. One can set the start and end day and month of a season, the bounds for GDD, the threshold of KDD, the number of temperature percentile bins, the threshold volume for what is considered a day with rain, and the length in years of what the user considers “long run.”

The command itself is agnostic in terms of which of the many weather metrics that it calculates a researcher chooses to use. However, in creating the **wxsum** command, we are not endorsing a theory-free or ad hoc approach to incorporating measures of weather into socioeconomic analysis. There is a growing body of literature that raises concerns regarding the use (and misuse) of weather data, particularly when weather is used as a source of identifying exogenous variation (Benami et al. 2026). This literature raises concerns about the way gridded weather data is matched with socioeconomic data (Michler et al. 2022), the existence of measurement error in Earth observation weather data Josephson et al. (2026), and the validity of the assumption that the weather is truly exogenous (Sarsons 2015; Mellon 2025). While incorporating weather data into one’s analysis is now low cost, a naive or unexamined use of that data will continue to have a high cost in terms of the accuracy of one’s analysis and results.

2 Data considerations

The **wxsum** command is designed for a common structure of remotely sensed or gridded weather products after they have been matched to locations in an analysis dataset. Each observation is a location, plot, household, firm, or other spatial unit, and each weather variable is a daily measurement for that unit. Thus the command begins after the spatial matching step: the user should first decide which weather product, temporal aggregation, and spatial linking rule are appropriate for the research design. These choices remain substantive, not merely computational, because estimates can be sensitive to the source of weather data and to the way gridded observations are linked to socioeconomic observations (Auffhammer et al. 2013; Donaldson and Storeygard 2016; Michler et al. 2022; Josephson et al. 2026).

2.1 Shape and naming convention

Input data must be in wide form. Daily weather values are stored in separate numeric variables whose names share a common prefix and end with an eight-digit date in `yyyymmdd` format. For example, if the precipitation variables are named with the

prefix `pic_`, then precipitation on May 15, 1979, should appear in a variable named `pic_19790515`. If temperature variables are named with the prefix `t_`, then temperature on the same date should appear in `t_19790515`. Variables not beginning with the specified prefix are ignored unless they are retained by using the `keep()` option.

The wide representation is convenient for the gridded weather extracts commonly used in applied work because the unit of analysis stays in one row and each daily exposure is represented by a dated variable. It can also produce large datasets when daily histories cover many years or many spatial units. Thus the practical constraint is Stata's ability to hold the matched weather extract in memory rather than an additional conceptual limit imposed by `wxsum`.

This naming convention makes the calendar explicit. The command parses the date suffix of each weather variable, identifies the days that belong to each user-defined season, and computes statistics separately for each location and season. Because all computations are based on the dates in variable names, users should check that the date suffixes are complete, unique, and measured on the intended calendar. The command can accommodate seasons that cross calendar years. For example, a season that begins in November and ends in February is assigned to the year in which the season begins.

2.2 Seasons, units, and missing values

A season is defined by an initial month and final month and, optionally, by an initial day and final day. This flexibility is useful because the relevant period for rainfall or heat exposure differs across crops, agroecological zones, and outcomes. A growing season may be May through October in one application, whereas a heat-exposure season may begin late in the prior calendar year in another. The command does not impose a biological or econometric interpretation on the chosen season; the user should define the exposure window before running the command and should report that choice in subsequent empirical work.

The command is similarly agnostic about units. Rainfall can be measured in millimeters, centimeters, or inches, and temperature can be measured in degrees Celsius or Fahrenheit, provided that all daily variables are measured consistently. Unit choices matter for thresholds. The lower and upper bounds for growing degree days and the base for killing degree days must be expressed in the same units as the daily temperature data. The rainy-day threshold must be expressed in the same units as the daily precipitation data.

Many weather products contain missing values because of sensor problems, spatial coverage, cloud contamination, or limitations in station interpolation. Before using `wxsum`, users should examine missingness in the daily variables and decide whether the weather product and temporal coverage are adequate for the application. The command calculates seasonal statistics from the available values in the selected window. If an empirical design requires a balanced panel of weather exposures or a minimum number of observed days within a season, that restriction should be imposed before or after running `wxsum` and should be documented in the replication files.

2.3 Long-run reference periods

Several variables generated by **wxsum** describe weather shocks as deviations from a location-specific long-run reference period. Let w_{it} be a seasonal weather statistic for location i in the season beginning in year t , and let L be the number of prior years requested in `lr_years()`. The rolling long-run mean and standard deviation used for deviations and z scores are based only on strictly preceding seasons:

$$\bar{w}_{it}^L = L^{-1} \sum_{l=1}^L w_{i,t-l}$$

$$s_{it}^L = \left\{ (L-1)^{-1} \sum_{l=1}^L (w_{i,t-l} - \bar{w}_{it}^L)^2 \right\}^{1/2}$$

The deviation is $w_{it} - \bar{w}_{it}^L$, and the corresponding z score is $(w_{it} - \bar{w}_{it}^L)/s_{it}^L$. The use of strictly prior years avoids including the current realization in its own benchmark. If there are too few preceding years to satisfy the requested value of `lr_years()`, the long-run deviations and z scores are not generated for those initial seasons, although the nonreference statistics are still calculated.

3 The wxsum command

3.1 Syntax

The command is installed from the development repository by typing

```
. local repo https://raw.githubusercontent.com/jdavidm/wxsum/master/
. net install wxsum, from("`repo`") replace
```

The syntax is

```
wxsum prefix , ini_month(month) fin_month(month) [ ini_day(day)
  fin_day(day) rain_data temp_data gdd_lo(num) gdd_hi(num)
  kdd_base(num) bins(num) lr_years(num) keep(varlist) save(filename)
  rain_threshold(num) ]
```

Here *prefix* is the common stub for the daily weather variables. It is not a varlist. If the data contain variables named `pic_19790101`, `pic_19790102`, and so on, the prefix supplied to the command is `pic_`. Exactly one of `rain_data` and `temp_data` should be specified.

3.2 Required variables

The minimal dataset contains an identifier for each location and a set of daily weather variables. The identifier is not an argument of **wxsum**; it is retained with `keep()` if it

should appear in the output dataset. The weather variables must be numeric and must use the naming convention described above. Table 1 summarizes these requirements.

Table 1: Data requirements for `wxsum`

Element	Requirement
Observation	One row per spatial or analytic unit.
Weather variables	One variable per day, with a common prefix and a <code>yyyymmdd</code> date suffix.
Date information	Encoded in the weather-variable names rather than stored in separate year, month, and day variables.
Rainfall data	Numeric daily precipitation variables; use <code>rain_data</code> .
Temperature data	Numeric daily temperature variables; use <code>temp_data</code> .
Other variables	Retain with <code>keep()</code> if identifiers, coordinates, or covariates should remain in the output.

3.3 Options

`ini_month(month)` specifies the initial month of the season. This option is required.

`fin_month(month)` specifies the final month of the season. This option is required. If the final month precedes the initial month in the calendar year, the command treats the season as crossing calendar years.

`ini_day(day)` specifies the first day of the initial month to be included in the season. The default is 01.

`fin_day(day)` specifies the last day of the final month to be included in the season. The default is 01.

`rain_data` specifies that the daily variables contain precipitation. This option is mutually exclusive with `temp_data`.

`temp_data` specifies that the daily variables contain temperature. This option is mutually exclusive with `rain_data`.

`gdd_lo(num)` specifies the lower bound for growing degree days. This option is required when `temp_data` is specified.

`gdd_hi(num)` specifies the upper bound for growing degree days. This option is required when `temp_data` is specified.

`kdd_base(num)` specifies the threshold above which killing degree days are calculated. The threshold should be in the same unit as the daily temperature variables.

`bins(num)` specifies the number of equal-sized temperature percentile bins to create. The default is 4, the minimum is 4, and the maximum is 10.

lr_years(num) specifies the number of strictly preceding years used to compute rolling deviations and z scores. The default is 10, and the maximum is 50.

keep(varlist) retains variables from the input dataset in the output dataset. This option is typically used for identifiers, administrative codes, coordinates, or other merge keys.

save(filename) saves the resulting dataset to *filename*. If **save()** is not specified, the generated variables remain in memory.

rain_threshold(num) specifies the threshold used to define a rainy day. The default is 1.

3.4 Output

wxsum is a data-management command rather than an estimation command. Its primary output is a dataset containing the retained variables requested in **keep()** and the generated seasonal weather variables. Because the command generates one set of variables for each season in the data, the number of output variables increases with the time span of the daily weather data. Generated variables are indexed by the season-start year. For example, if a season begins in November 2001 and ends in February 2002, the resulting seasonal statistics are associated with 2001, not 2002.

When **rain_data** is specified, the command produces the precipitation statistics in table 2. These variables combine simple summaries of the daily distribution, summaries of monthly totals, seasonal totals, rainy-day counts, and shock measures relative to the rolling long-run reference period.

Table 2: Rainfall statistics generated by **wxsum**

Group	Generated statistics
Daily distribution	Mean, median, standard deviation, and skewness of daily precipitation within the season.
Monthly totals	Mean, median, standard deviation, and skewness of monthly precipitation totals within the season.
Seasonal total	Total precipitation in the season, deviation from the long-run average of total precipitation, and z score of total precipitation.
Rainy days	Number of rainy days, number of days without rain, deviation from the long-run average number of rainy days, and deviation from the long-run average number of days without rain.
Rainy-day share	Percentage of days with rain and deviation from the long-run average percentage of days with rain.
Dry spells	Longest intra-seasonal dry spell.

When **temp_data** is specified, the command produces the temperature statistics in table 3. The growing degree day calculation uses **gdd_lo()** and **gdd_hi()** as lower and

upper bounds, whereas killing degree days are calculated relative to `kdd_base()`. Temperature bins report the percentage of seasonal days falling into the requested number of equal-sized percentile bins.

Table 3: Temperature statistics generated by `wxsum`

Group	Generated statistics
Daily distribution	Mean, median, standard deviation, skewness, and maximum daily temperature within the season.
Growing degree days	Growing degree days, deviation from the long-run average of growing degree days, and z score of growing degree days.
Killing degree days	Killing degree days, deviation from the long-run average of killing degree days, and z score of killing degree days.
Temperature bins	Percentage of days falling in each requested percentile bin.

The command calculates the nonreference variables whenever the selected daily variables and season are available. Rolling deviations and z scores require sufficient prior history at the same location. Consequently, if the data begin in 2000 and the user requests `lr_years(10)`, the first eligible long-run deviations are not available until the season beginning in 2010. This behavior is intentional: the current season is never included in the reference period against which it is compared.

4 Examples

The repository includes small rainfall and temperature datasets that can be used to check installation and illustrate the syntax. These files are also useful as templates for checking the required wide layout and variable-naming convention. The examples below begin by loading those datasets, as they should in a reproducible do-file.

4.1 Rainfall summaries

Suppose that the daily rainfall variables in `rain.dta` are named with the prefix `pic_`. The following commands calculate seasonal rainfall statistics for a May 15–October 15 growing season and save the resulting dataset:

```
. use rain.dta, clear
. wxsum pic_, ini_month(05) fin_month(10) ini_day(15) ///
>   fin_day(15) rain_data lr_years(10) rain_threshold(1) ///
>   save(rainfall_stats.dta)
```

The output dataset contains one observation for each input location. For each season observed in the daily rainfall variables, `wxsum` adds the rainfall statistics listed in table 2. If the user also needs location identifiers or other merge variables, these should be supplied in `keep()`:

```
. use rain.dta, clear
. wxsum pic_, ini_month(05) fin_month(10) ini_day(15) ///
>   fin_day(15) rain_data keep(id region) ///
>   save(rainfall_stats.dta)
```

The resulting file can then be merged to a socioeconomic dataset by the retained identifier variables and season-specific generated weather variables.

4.2 Temperature summaries

For temperature data, the user specifies `temp_data` and supplies the growing-degree-day bounds. The next example uses a season that begins in November and ends in February. Because the final month precedes the initial month, the command treats the exposure window as spanning calendar years and assigns the season to the year in which it begins.

```
. use temp.dta, clear
. wxsum t_, ini_month(11) fin_month(02) temp_data ///
>   gdd_lo(8) gdd_hi(32) kdd_base(32) bins(4) ///
>   keep(id region)
```

This command generates the temperature summaries listed in table 3. The values `gdd_lo(8)`, `gdd_hi(32)`, and `kdd_base(32)` are illustrative; they should be chosen to match the units of the underlying temperature data and the biological or behavioral process being studied.

4.3 Changing the long-run benchmark

The default long-run reference period is 10 prior years. Researchers working with longer weather histories may prefer a longer benchmark, whereas applications with short panels may require a shorter one. The following command uses the prior 20 years as the rolling reference period:

```
. use rain.dta, clear
. wxsum pic_, ini_month(05) fin_month(10) rain_data ///
>   lr_years(20) keep(id)
```

Changing `lr_years()` affects only the rolling deviations and z scores. It does not affect the seasonal means, medians, standard deviations, skewness measures, totals, counts, or dry-spell measures.

5 Conclusions

`wxsum` reduces the amount of custom code needed to transform daily weather data into seasonal measures commonly used in socioeconomic analysis. It standardizes the construction of summary statistics, threshold-based heat measures, rainy-day counts, dry spells, and rolling weather-shock measures while leaving the substantive choices—data source, spatial match, season definition, units, and thresholds—with the researcher. This division of labor is useful: the command lowers the probability of programming errors

in repetitive weather processing tasks, but it does not substitute for careful research design.

Future development could extend `wxsum` in several directions. First, additional output formats could be useful for users who prefer long datasets with one row per location-season rather than wide datasets with one variable per statistic-year. Second, diagnostic options that report missing days, season length, and the number of prior years available for each generated shock measure would make quality control easier in large applications. Third, the command could be expanded to additional weather and climate variables, such as humidity, wind speed, vapor pressure deficit, or solar radiation, when these variables are available in daily gridded products. These extensions would preserve the main purpose of the command: to make weather-data processing in Stata more transparent, reproducible, and less error prone.

6 References

- Auffhammer, M., S. Hsiangy, W. Schlenker, and A. Sobelz. 2013. Using Weather Data and Climate Model Output in Economic Analyses of Climate Change. *Review of Environmental Economics and Policy* 7(2): 181–198.
- Benami, E., J. D. Michler, A. Josephson, M. Cecil, and G. Maskell. 2026. The Uses (and Misuses) of Earth Observation Data on Weather and Vegetation. In *Geospatial Impact Evaluation in Practice*, ed. A. BenYishay and K. Singh. Taylor & Francis.
- Dell, M., B. F. Jones, and B. A. Olken. 2014. What Do We Learn from the Weather? The New Climate-Economy Literature. *Journal of Economic Literature* 52(3): 740–798.
- Donaldson, D., and A. Storeygard. 2016. The View from Above: Applications of Satellite Data in Economics. *Journal of Economic Perspectives* 30(4): 171–98.
- Josephson, A., J. D. Michler, T. Kilic, and S. Murray. 2026. The mismeasure of weather: Using earth observation data for estimation of socioeconomic outcomes. *Journal of Development Economics* 178: 103553.
- Mellon, J. 2025. Rain, Rain, Go Away: 194 Potential Exclusion-Restriction Violations for Studies Using Weather as an Instrumental Variable. *American Journal of Political Science* 69(3): 881–898.
- Michler, J. D., A. Josephson, T. Kilic, and S. Murray. 2021. Estimating the Impact of Weather on Agriculture. World Bank Policy Research Working Paper, No. 9867.
- . 2022. Privacy protection, measurement error, and the integration of remote sensing and socioeconomic survey data. *Journal of Development Economics* 158: 102927.
- Sarsons, H. 2015. Rainfall and Conflict: A Cautionary Tale. *Journal of Development Economics* 115: 62–72.

About the authors

Jeffrey D. Michler is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Anna Josephson is an Associate Professor in the Department of Agricultural and Applied Economics at the University of Arizona.

Aleksandr Michua is a Visiting Assistant Professor at the Department of Economics at Swarthmore College.

Oscar Barriga Cabanillas is an Economist at the World Bank.

Jeffrey C. Oliver is a Data Science Specialist in the University of Arizona Libraries.